Augmented Reality with ArUco Markers

Hochschule Ravensburg-Weingarten

[1em]

Jonas Alber
Master Informatik
15444427
jonas.alber@hs-weingarten.de

Tim Schweitzer
Master Informatik
29844429
tim.schweitzer@hs-weingarten.de

December 29, 2024

Disclaimer

The content of this document was created by the authors mentioned above. ChatGPT from OpenAI was used to improve readability. However, all content has been checked by the authors and adapted where necessary.

List of abbreviations

KITTI Karlsruhe Institute of Technology and Toyota Technological Institute

YOLO You only look once

Bbox Bounding Box

CNN Convolutional Neural Networks

1 Introduction

2 Setup

2.1 Neuronal Network

YOLO (You Only Look Once) is an object detection system that was first released in 2016 [**?**]. It is required for this project. The YOLOv11 model was used for object detection in the KITTI dataset.

Erklären kurz wieso wir YOLOv11 nutzen und das YOLO ein CNN ist

2.2 Preprocessing

To utilize the Karlsruhe Institute of Technology and Toyota Technological Institute (KITTI) dataset for our object detection and depth estimation task, we prepared the data by resizing images and converting the annotation format. This step ensures compatibility with the You only look once (YOLO)v11 object detection framework. The preprocessing script, implemented in Python, performs the following tasks:

- Annotation Conversion.
- Image Resizing.

The script processes the dataset once, converting and saving the images and annotations in the required format.

2.2.1 Annotation Conversion

The images used define the Bounding Box (Bbox) using the KITTI standard. This definition is not compatible with the Bbox definition of YOLO, so they need to be converted. The KITTI defines the Bbox with the following parameters:

- `x_min, y_min`: Upper left corner coordinates of the image.
- `x_max, y_max`: Lower right corner coordinates of the image.

so that the resulting Bbox looks like:

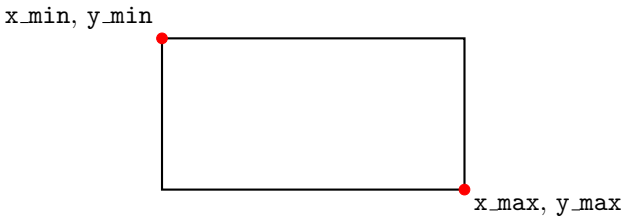

Figure 1: Example of a bounding box in KITTI format.

In contrast, the YOLO format specifies each bounding box as:

- `class_id`: Numeric identifier for the object class.
- `x_center, y_center`: Normalized coordinates of the bounding box center.
- `width, height`: Normalized dimensions of the bounding box.

KITTI does not support the class identifier, so we omitted this field in the conversion. Therefore, car is identified as 1. To convert the bounding box coordinates, we used the following formulas:

$$\texttt{img\_width} = x_{max} - x_{min}, \quad \texttt{img\_height} = y_{max} - y_{min}$$

$$x\_center = \frac{x_{min} + x_{max}}{2 \cdot \texttt{img\_width}}, \quad y\_center = \frac{y_{min} + y_{max}}{2 \cdot \texttt{img\_height}}$$

$$width = \frac{x_{max} - x_{min}}{\texttt{img\_width}}, \quad height = \frac{y_{max} - y_{min}}{\texttt{img\_height}}$$

The resulting Bbox in YOLO format is shown in Figure 2.

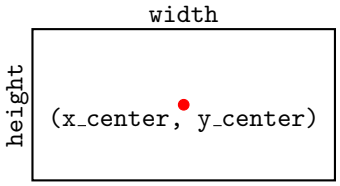

Figure 2: Example of a bounding box in YOLO format.

2.2.2 Image Resizing

To standardize input dimensions for the object detector, all images were resized to 1248 × 512 pixels using OpenCV. The resized images were stored in the `images/` subdirectory of the output path.

Erklären, warum wir diese Auflösung verwenden: Die Auflösung 1248 × 512 Pixel wurde gewählt, um ein Gleichgewicht zwischen Recheneffizienz und ausreichenden Details für eine genaue Objekterkennung zu gewährleisten. Diese Auflösung ist auch mit den Eingabeanforderungen des YOLOv11-Modells kompatibel, was eine optimale Leistung während des Trainings und der Inferenz sicherstellt.

3 Training

3.1 Training Configuration

We split the dataset into 80% for training and 20% for validation, resulting in 16 images for training and 4 images for validation. This 80:20 split is a common practice in training Convolutional Neural Networks (CNN) as it provides a good balance between having enough data to train the model and enough data to validate its performance.

The training configuration included the following key hyperparameters: The model was trained for 100 epochs. An epoch refers to one complete pass through the entire training dataset. Training for multiple epochs helps the model to learn and generalize better. A batch size of 32 was used. This means that 32 samples from the training dataset were processed before the model's internal parameters were updated. A larger batch size can lead to more stable gradient updates. Data augmentation was enabled (set to True). Data augmentation involves applying random transformations to the training data, such as rotations, translations, and flips, to artificially increase the diversity of the training dataset and improve the model's robustness.

3.2 Training Process

Erklären die ergebnisse unseres Trainings mit Bildern und Werten.

4 Training Evaluation

Using the trained YOLOv11 model, predictions were generated for test images. The model outputs bounding boxes, confidence scores, and class labels for each detected object. The process is implemented as follows:

Listing 1: Generating predictions using YOLOv11

```
# Load the trained YOLOv11 model
load_model(model_path)

# Predict on a sample image
results = model.predict(image_path)

# Process the prediction results
for each result in results:
    extract bounding box coordinates
    extract confidence scores
    extract class labels
    calculate IoU
```

Was bringt der Code überhaupt in der Count Doku?

To evaluate the quality of predictions, the Intersection over Union (IoU) metric was calculated for each detected bounding box. The IoU is defined as:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

where the overlap is the intersection of the predicted and ground truth bounding boxes.

The Intersection over Union (IoU) metric is calculated as follows:

Besseres Beispiel für IoU

Given two bounding boxes, modelPredictionBox = (x_1, y_1, x_2, y_2) and expectedBox = $(x_{1g}, y_{1g}, x_{2g}, y_{2g})$:

1. Calculate the coordinates of the intersection rectangle:

$$x_i1 = \max(x_1, x_{1g}), \quad y_i1 = \max(y_1, y_{1g})$$

$$x_i2 = \min(x_2, x_{2g}), \quad y_i2 = \min(y_2, y_{2g})$$

2. Calculate the area of the intersection rectangle:

$$\text{inter\_area} = \max(0, x_i2 - x_i1) \times \max(0, y_i2 - y_i1)$$

3. Calculate the area of both bounding boxes:

$$\text{box1\_area} = (x_2 - x_1) \times (y_2 - y_1)$$

$$\text{box2\_area} = (x_{2g} - x_{1g}) \times (y_{2g} - y_{1g})$$

4. Calculate the union area:

$$\text{union\_area} = \text{box1\_area} + \text{box2\_area} - \text{inter\_area}$$

5. Finally, calculate the IoU:

$$\text{IoU} = \frac{\text{inter\_area}}{\text{union\_area}}$$

5 Distance Estimation

Hilfe, was ist das?

Using the intrinsic matrix provided in the KITTI dataset, we estimated the horizontal distance to objects based on their bounding box coordinates. The procedure involves:

1. Transforming pixel coordinates (bounding box center) to camera coordinates using the intrinsic matrix.
2. Projecting a ray from the camera through the pixel into the 3D environment.
3. Intersecting this ray with the ground plane to calculate the distance.

The calculation is described mathematically as follows:

$$\mathbf{L} = K^{-1} \cdot \mathbf{p}, \quad \mathbf{p} = [x, y, 1]^T$$

$$t = \frac{\text{camera height}}{-\text{ray direction}_y}, \quad \text{intersection} = t \cdot \mathbf{L}$$

The Python implementation is shown below:

Listing 2: Distance calculation

```
def calculate_distance_from_bbox(predicted_boxes, img_height, img_width, K, camera_height):
    for box in predicted_boxes:
        x_min, y_min, x_max, y_max = box
        x_center = (x_min + x_max) / 2
        y_center = y_max
# Bottom of the box

        # Transform to camera coordinates
        ray_direction = np.linalg.inv(K) @ np.array([x_center, y_center, 1])
        ray_direction /= np.linalg.norm(ray_direction)

        # Scale to ground plane
        t = camera_height / -ray_direction[1]
        intersection = t * ray_direction
        horizontal_distance = np.linalg.norm(intersection[:2])
        print(f"Horizontal Distance: {horizontal_distance:.2f} meters")
```

Figure 3: Predicted bounding boxes (green) and ground truth (red) with calculated distances.

5.1 Results

6 Discussion

7 Conclusion